Bridge Day 8 INSTRUCTOR KEY

Debugging, Boundary Tests, and Complete Repair Evidence

Wednesday, July 15, 2026 • 20 points • target 17 minutes

Name		UIN		Section	

Assessment boundary and evidence source

Contract: 07a04826c975
Cumulative foundations: input conversion, comparisons, Boolean repair, expected vs actual, boundary tests
Scaffold level: complete program plus protective test set
Evidence source: the matching v5 workbook readiness/evidence pages and VIP activities 18.13, 18.14, 18.15, 18.16, 18.17. Show intermediate state for trace credit. Program credit requires one coherent solution, a stated assumption when the prompt leaves one implicit, and at least one test whose expected result can distinguish correct from incorrect logic.

Item 1. Diagnose an inclusive-boundary defect. - 5 points

```
temperature = 35
ready = temperature > 15 and temperature < 35
print(ready)
```

Question: The requirement includes 15 and 35. Give actual result, cause, corrected assignment, and two tests that expose the defect.

Model response: The actual result at 35 is False because temperature < 35 excludes equality. Correct with ready = 15 <= temperature <= 35. Tests 15 -> True and 35 -> True expose both strict-boundary defects; 14.9 and 35.1 should be False.

Item 2. Build a minimum protective boundary set. - 5 points

```
reading = 10
low = 10
high = 20

if reading < low:
    status = "below"
elif reading > high:
    status = "above"
else:
    status = "within"

print(status)
```

Question: Give exact supplied output, then specify four tests that protect both inclusive boundaries and their immediate outside cases.

Model response: The supplied output is within. A strong four-test set is 9.9 -> below, 10 -> within, 20 -> within, and 20.1 -> above. Each expected result targets a distinct comparison boundary.

Complete program - 10 points

- Read reading, low, and high as floats.
- Print invalid when low is greater than high.
- Otherwise print Result: below, Result: within, or Result: above. Equality at either limit is within.
- Use one mutually exclusive chain and provide four boundary tests with expected output.

Program workspace

Model program

```
reading = float(input())
low = float(input())
high = float(input())

if low > high:
    print("invalid")
elif reading < low:
    print("Result:  below")
elif reading > high:
    print("Result:  above")
else:
    print("Result:  within")
```

Test input, expected result, and why this test matters

Reasoning and test evidence The limit-order guard comes first. The below and above branches use strict comparisons, leaving equality at either boundary for within. For low 10 and high 20, use 9.9 -> Result: below, 10 -> Result: within, 20 -> Result: within, and 20.1 -> Result: above. A reversed pair such as low 20 and high 10 must print invalid.
Scoring guidance: 2 input/limit validation, 3 correct ordered classifier, 2 inclusive-boundary behavior, 1 exact output, 2 four-case test evidence.

Quiz evidence is sampled from the taught construct boundary; workbook and notebook evidence remain the primary learning surfaces.